CS7643 Assignment 1

Tyler Crosse
Georgia Tech
tcrosse3@gatech.edu

1 Theory Problem Set

1.1 Theory Q1

The softmax function $s(z)$, which takes a vector input z and outputs a vector whose ith entry s_i is:

$$s_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \tag{1}$$

The input vector z to $s(\cdot)$ is sometimes called the *logits*, which just means the unscaled output of previous layers. Derive the gradient of s with respect to the logits, i.e., derive $\frac{\partial s}{\partial z}$. Consider re-using your work from PS0.

The softmax function $\mathbf{f}$ takes in the vector $\mathbf{z} \in \mathbb{R}^n$ and returns a scaled version of it $\mathbf{s} \in \mathbb{R}^n$, so $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^n$. Since these are both vectors the gradient of $\mathbf{s}$ w.r.t. $\mathbf{z}$ can be described by the Jacobian $\mathbf{J} = \nabla_{\mathbf{z}}\mathbf{f} = \frac{\mathbf{df(x)}}{\mathbf{z}} = \frac{\partial \mathbf{s}}{\partial \mathbf{z}} \in \mathbb{R}^{n \times n}$.

Writing out the Jacobian matrix in more detail helps makes the indices more explicit.

$$\mathbf{J} = \frac{\partial \mathbf{s}}{\partial \mathbf{z}} = \begin{bmatrix} \frac{\partial s_1}{\partial z_1} & \frac{\partial s_1}{\partial z_2} & \cdots & \frac{\partial s_1}{\partial z_n} \\ \frac{\partial s_2}{\partial z_1} & \frac{\partial s_2}{\partial z_2} & \cdots & \frac{\partial s_2}{\partial z_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial s_n}{\partial z_1} & \frac{\partial s_n}{\partial z_2} & \cdots & \frac{\partial s_n}{\partial z_n} \end{bmatrix}$$

So, the i-th row and j-th column in the Jacobian matrix $J_{ij} = \frac{\partial s_i}{\partial z_j}$

This partial for a single cell of J_{ij} can be found using the quotient rule. Let

$$h(x) = \frac{f(x)}{g(x)}$$

where both f and g are differentiable and $g(x) \neq 0$.

$$\frac{dh}{dx} = \frac{d}{dx}\Big[\frac{f(x)}{g(x)}\Big] = \frac{g(x)\frac{df}{dx} - f(x)\frac{dg}{dx}}{(g(x))^2}$$

then

$$h'(x) = \frac{f'(x)g(x) - f(x)g'(x)}{(g(x))^2}$$

Let $f = e^{z_i}$ and $g = \sum_k e^{z_k}$

Referencing the Jacobian matrix, on the diagonal when $i = j$, then $f' = \frac{\partial f}{z_j} = \frac{\partial f}{z_i} = e^{z_i}$ and $g' = \frac{\partial g}{z_j} = \frac{\partial g}{z_i} = e^{z_k}$

$$\frac{\partial s_i}{\partial z_j} = \frac{e^{z_i} \sum_k e^{z_k} - e^{z_i} e^{z_k}}{(\sum_k e^{z_k})^2}$$

with some factoring

$$= \frac{e^{z_i}}{\sum_k e^{z_k}} \Big(1 - \frac{e^{z_i}}{\sum_k e^{z_k}}\Big)$$

and by how s_i was defined

$$= s_i(1 - s_i)$$

When $i \neq j$, then $f' = \frac{\partial f}{z_j} = 0$ (because it's w.r.t z_j) and $g' = \frac{\partial g}{z_j} = e^{z_k}$

$$\frac{\partial s_i}{\partial z_j} = \frac{0 \cdot \sum_k e^{z_k} - e^{z_i} e^{z_j}}{(\sum_k e^{z_k})^2}$$

with a bit of factoring

$$= -\frac{e^{z_i}}{\sum_k e^{z_k}} \frac{e^{z_i}}{\sum_k e^{z_j}}$$

and applying the definition of s_i

$$= s_i s_j$$

So for the two cases

$$\frac{\partial s_i}{\partial z_j} = \begin{cases} s_i \left(1 - s_i\right) & \text{if } i = j \\ -s_i \, s_j & \text{if } i \neq j \end{cases}$$

Which defines the full Jacobian matrix of the gradient s with respect to the logits.

$$\frac{\partial \mathbf{s}}{\partial \mathbf{z}} = \begin{bmatrix} s_1(1 - s_1) & -s_1 s_2 & \cdots & -s_1 s_n \\ -s_2 s_1 & s_2(1 - s_2) & \cdots & -s_2 s_n \\ \vdots & \vdots & \ddots & \vdots \\ -s_n s_1 & -s_n s_2 & \cdots & s_n(1 - s_n) \end{bmatrix}$$

References for Jacobians, the quotient rule and Latex formatting:

- The Matrix Calculus You Need For Deep Learning 2018 by Terrence Parr and Jeremy Howard [1]
- Mathematics for Machine Learning 2020 by Marc Peter Deisenroth, A. Aldo Faisal, Cheng Soon Ong [2]
- https://tex.stackexchange.com/questions/204621/matrix-in-latex [3]

1.2 Theory Q2

Implement AND and OR for pairs of binary inputs using a single linear threshold neuron with weights $w \in \mathbb{R}^2$, bias $b \in \mathbb{R}$, and $x \in \{0,1\}^2$:

$$f(x) = \begin{cases} 1 & \text{if } w^T x + b \geq 0 \\ 0 & \text{if } w^T x + b < 0 \end{cases} \tag{2}$$

That is, find w_{AND} and b_{AND} such that:

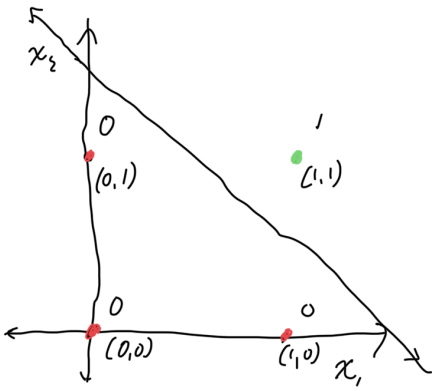
x_1	x_2	$f_{\text{AND}}(x)$
0	0	0
0	1	0
1	0	0
1	1	1

Also, find w_{OR} and b_{OR} such that:

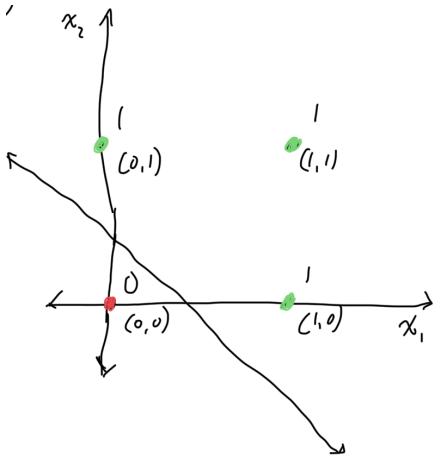
x_1	x_2	$f_{\text{OR}}(x)$
0	0	0
0	1	1
1	0	1
1	1	1

To separate the values of AND, any line that $\geq (1,0)$ and $(0,1)$ while $\leq (1,1)$ works. A simple one is $x_2 = -x_1 + 1.5 \implies 0 = x_1 + x_2 - 1.5$ so $w = [1,1]$ and $b = -1.5$

The approach to separate the values of OR is very similar. Any line that $\leq (1,0)$ and $(0,1)$ while $\geq (0,0)$ works. A simple one is $x_2 = -x_1 + 0.5 \implies 0 = x_1 + x_2 - 0.5$ so $w = [1,1]$ and $b = -0.5$



(a) AND $x_2 = -x_1 + 1.5$



(b) OR $x_2 = -x_1 + 0.5$

Figure 1: Linear Separations for AND, OR

1.3 Theory Q3

Consider the XOR function:

x_1	x_2	$f_{\text{XOR}}(x)$
0	0	0
0	1	1
1	0	1
1	1	0

Prove that XOR can NOT be represented using a linear model with the same form as (2). [Hint: To see why, plot the examples from above in a plane and think about drawing a linear boundary that separates them.]

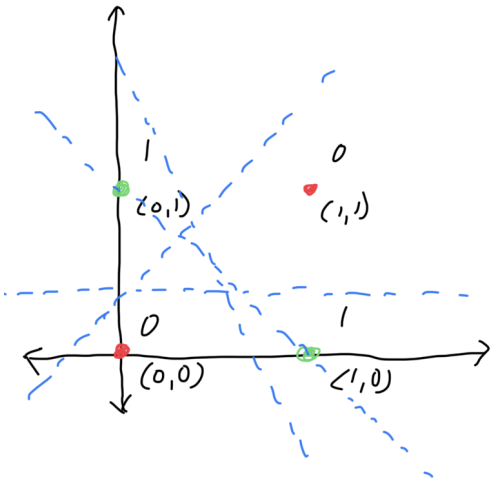


Figure 2: No way to linearly separate XOR

There isn't a way to linearly separate the points of XOR.

A more formal proof can be shown by contradiction. To separate the values, a linear model would need:

$$f(x_1, x_2) = \begin{cases} 1 & \text{if } w_1 x_1 + w_2 x_2 + b \geq 0 \\ 0 & \text{else} \end{cases}$$

Breaking this apart:

1. $f(0,0) = 0 \implies b < 0$
2. $f(0,1) = 1 \implies w_2 + b \geq 0$
3. $f(1,0) = 1 \implies w_1 + b \geq 0$
4. $f(1,1) = 0 \implies w_1 + w_2 + b < 0$

(2) and (3) can be rewritten as $w_2 \geq -b$ and $w_1 \geq -b$ which implies $w_1 + w_2 \geq -2b \implies w_1 + w_2 + 2b \geq 0$. This contradicts (4) $w_1 + w_2 + b < 0$. Therefore, XOR is not linearly separable ■.

2 Paper Review

'Weight Agnostic Neural Networks' by Adam Gaier and David Ha

2.1 Paper Q1

What is the main contribution of this paper? In other words, briefly summarize its key insights. What are some strengths and weaknesses of this paper?

The paper introduces Weight Agnostic Neural Networks (WANNs), an approach that prioritizes architecture over weight optimization in neural networks. It demonstrates that certain architectures can encode solutions to tasks without extensive weight training. This work was motivated by observations of neuroscientists that the connectivity of neurons in the brain is highly structured. The WANN approach outlined uses a single weight for all neurons to reduce the search space by limiting the curse of dimensionality caused by trying to train N-weights. By using a shared weight parameter sampled from a random distribution, the authors identify network topologies with strong inductive biases that perform well across tasks like reinforcement learning and MNIST classification. This paper's biggest strengths are its novelty and the simplicity of the solution. The approach it introduces is a big challenge to focus purely on weight optimization. The weaknesses are that it cannot achieve STOA, and how well it will scale to much more complex tasks is unclear.

2.2 Paper Q2

What is your personal takeaway from this paper? This could be expressed either in terms of your perceived novelty of this paper compared to others you've read in the field, potential future directions of research in the area that the authors haven't addressed, or anything else that struck you as being noteworthy.

My biggest takeaways were that weights aren't everything and that there is a lot of inductive bias in the architecture of a network. The release of the full Connectome of Drosophila (the common fruit fly) in October 2024 blew me away. Research into the fly's brain revealed many physical actions, like fleeing, that were triggered in predictable ways by activating only a few physical neurons. This blurred my intuition on what constitutes thought. The WANN paper seems to apply a similar perspective to artificial neural networks. It aligns with the idea that architecture search is as critical as weight optimization, inspiring new ways to design neural networks with innate biases. Future research could explore hybrid methods combining WANNs with traditional gradient-based optimization, scaling WANNs to complex tasks, or using them for transfer learning. Integrating biologically inspired features, such as adaptive topology or plasticity, could further enhance the applicability of weight-agnostic networks. The potential connection back to biological neuron architectures is deeply interesting and inspiring.

2.3 Paper Q3

The traditional view of optimization in deep learning (and often in general) is that we are searching the space of weights to find the best ones. In other words, learning is a search problem. How would you view the above paper from the perspective of search?

This paper reframes learning as a search problem by shifting the focus from optimizing weights to discovering robust architectures. By evaluating architectures with a single shared random weight, the search process identifies topologies with strong inductive biases, effectively collapsing the search space and prioritizing structural robustness. This approach highlights an intriguing two-tiered view of optimization: first, searching for architectures with innate task-relevant properties, and second, optionally fine-tuning weights to improve performance. The focus on multi-objective search, balancing simplicity and functionality, aligns with principles like minimal description length and suggests efficient and interpretable architectures.

2.4 Paper Q4

One of the key aspects of deep learning is that, given a parameterized function, we can find weights to represent any function if it has sufficient depth and complexity. What does this paper say about the representational power of architectures given a fixed method for determining weights? Does the method for determining the weights matter? Do you think these two have equal representational power? Why or why not?

This paper highlights that neural network architectures possess intrinsic representational power independent of weight optimization. Focusing on "weight-agnostic" networks demonstrates that certain topologies can encode solutions for specific tasks even with randomly assigned or shared weights. This suggests that architecture is critical in determining a network's ability to approximate and represent functions, providing inductive biases that align with particular problems. While the paper shows that tuning weights can refine performance, it emphasizes the foundational role of topology in shaping the network's capacity to represent functions.

Architecture and weights play complementary roles in representation. Architecture defines the inductive biases and capacity for function approximation, while weights enable fine-tuning to fit specific data distributions. WANNs demonstrate that architecture can partially substitute for weights but not fully replace their role.

3 Writeup: Two-Layer Neural Network

3.1 Learning Rate Data

Learning Rate (lr)	**Training Accuracy (%)**	**Val. Accuracy (%)**	**Test Accuracy (%)**
1	**0.9452**	**0.9519**	**0.9489**
1e-1	0.9423	0.9363	0.9488
5e-2	0.9088	0.9142	0.9133
1e-2	0.7306	0.7748	0.7608

Table 1: Training, Validation, and Test Accuracy for Different Learning Rates

A learning rate of 1 resulted in the highest test accuracy in the learning rate tuning.

3.2 Learning Rate Plots

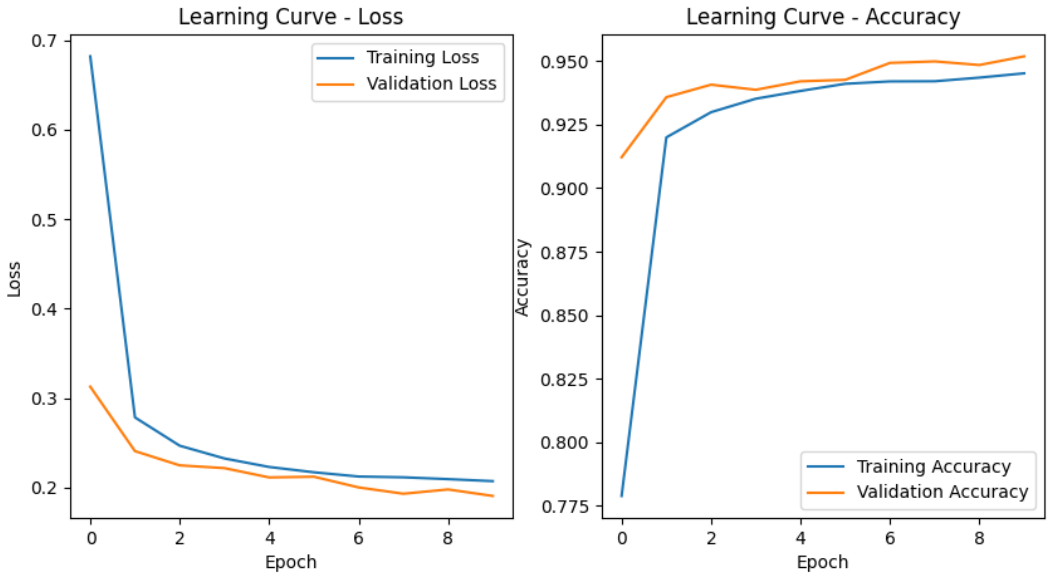


Figure 3: Learning Rate = 1

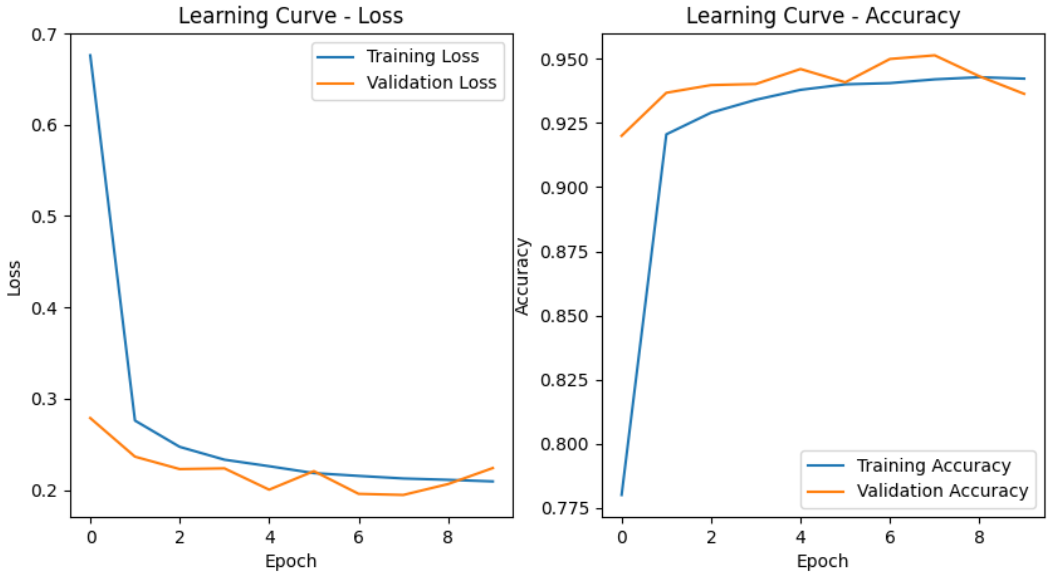


Figure 4: Learning Rate = 0.1

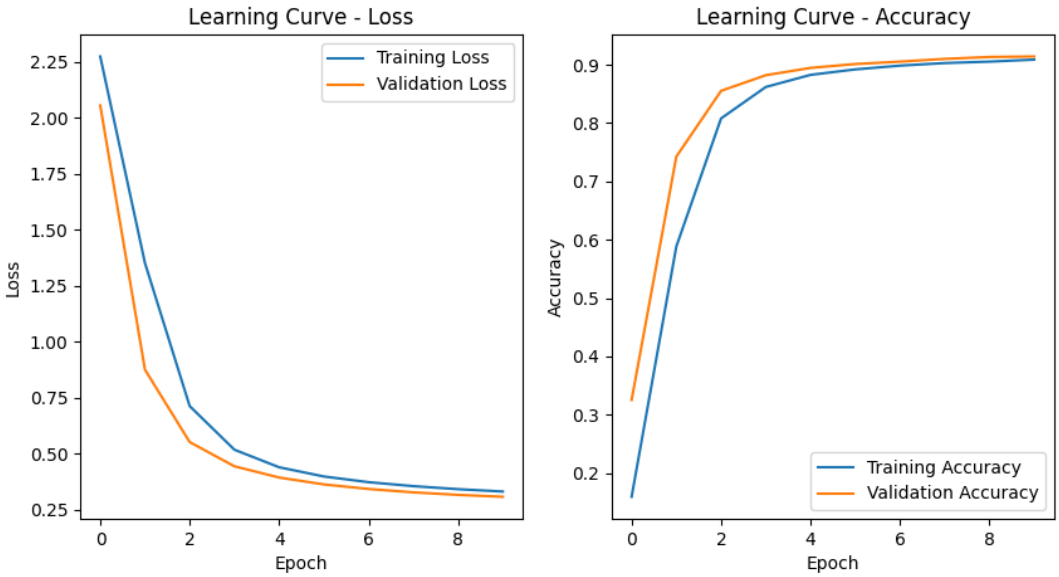


Figure 5: Learning Rate = 0.05

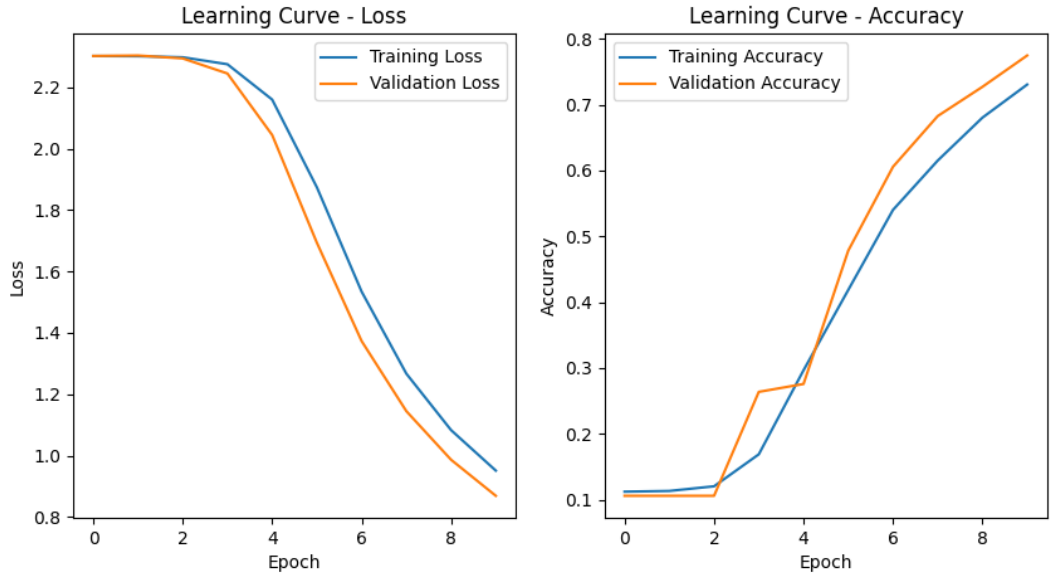


Figure 6: Learning Rate = 0.01

Learning Rate Discussion

Learning rate directly affects how the weights of the model are updated. Learning rates that are too slow increase the compute (wall clock time) needed for a model to converge. They can also cause a model to get stuck in local minima in the loss surface. Learning rates that are too fast can cause a model to overshoot global minima and can cause oscillations as it tries to go back and forth near a minimum.

In this experiment, the value of the learning rate was varied from 1 to 1e-2 while the other hyperparameters were held constant. The 2-layer model was trained for 10 epochs on MINST, an 80/20 train/validation split, a batch size of 64, 1e-3 L2 regularization, 0.9 for momentum, and a hidden layer size of 128. The training, validating, and test cross-entropy accuracy was observed.

The model trained with a learning rate of 1 had the highest of all three accuracies, and there was a clear trend towards higher accuracies as the learning rate decreased. The model with the lowest learning rate tested, 1e-2, had the lowest accuracy.

However, the learning curves suggest that the results are less clear. The slope of the LR1 model was the steepest, and the LR1e-2 model was the most gradual, empirically supporting the theory that lower learning rates reduce the speed of convergence. Yet the plots on the slower learning models are much smoother than the faster ones, which jump around a fair amount.

Extrapolating the slope suggests that the accuracy of the higher learning rate models started to plateau at the end of 10 epochs, while the training of the lower learning rate models was cut short before they were able to converge. Similarly, because not all of the models converged in 10 epochs, it is hard to evaluate bias and variance. The training and validation curves are fairly close for all of the models. The impact of the learning rate over more epochs was further explored in the more complete hyperparameter tuning section below.

3.3 Regularization Data

Alpha (α)	**Training Accuracy (%)**	**Validation Accuracy (%)**	**Test Accuracy (%)**
1	0.1047	0.1060	0.1028
1e-1	0.3389	0.2638	0.3557
1e-2	0.8844	0.8935	0.8936
1e-3	0.9214	0.9244	0.9233
1e-4	**0.9300**	**0.9345**	**0.9336**

Table 2: Training, Validation, and Test Accuracy for Different Regularization Coefficients

An α of 1e-4 resulted in the highest test accuracy in the regularization tuning.

3.4 Regularization Plots

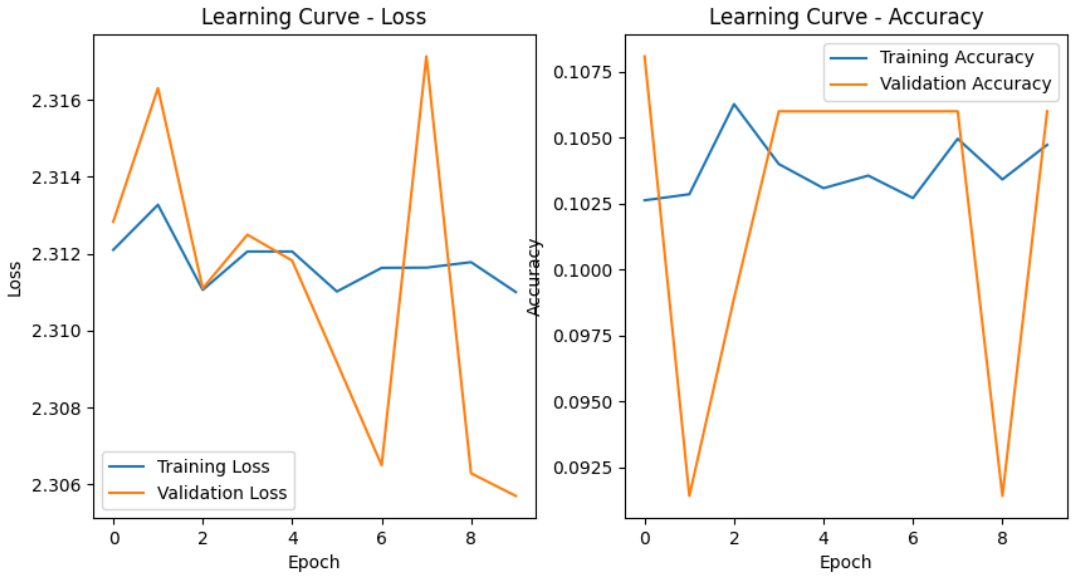


Figure 7: Alpha = 1

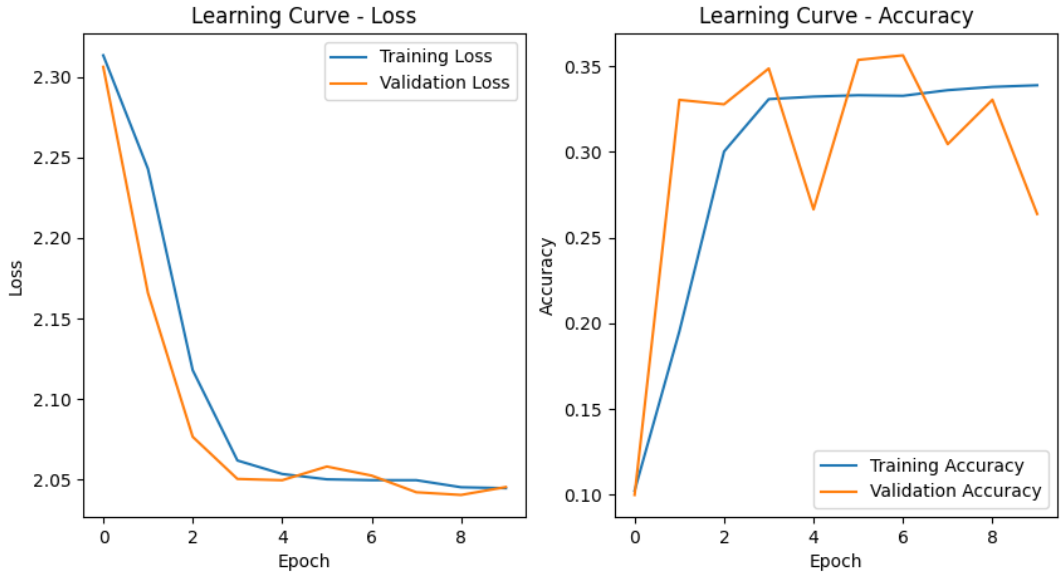


Figure 8: Alpha = 0.1

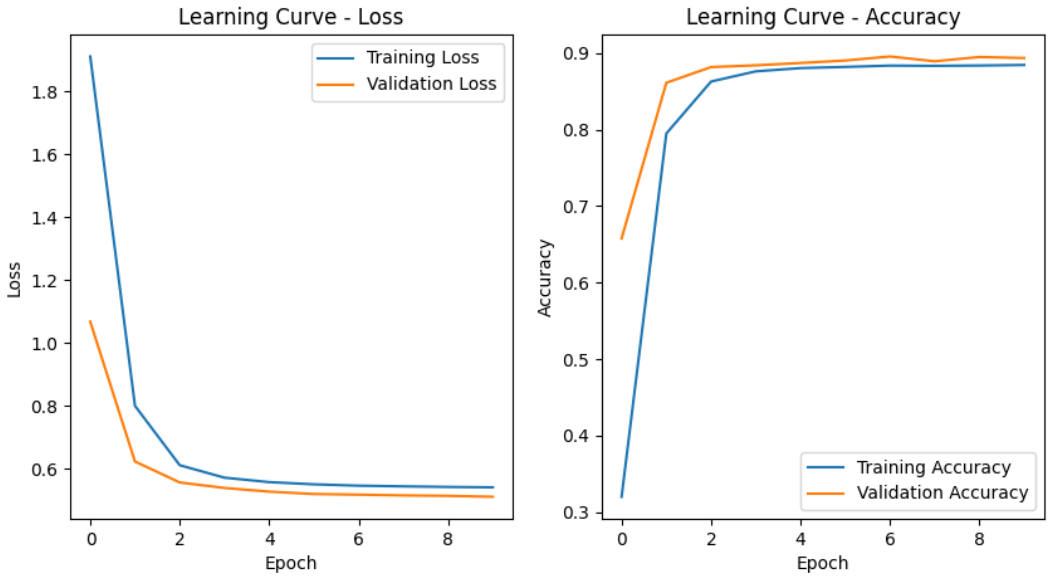


Figure 9: Learning Rate = 0.01

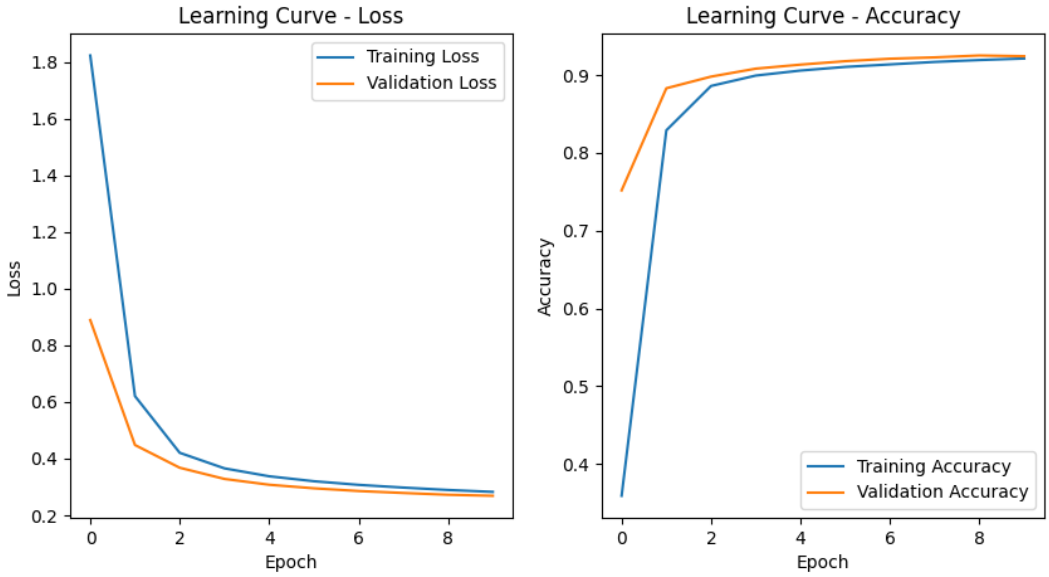


Figure 10: Alpha = 0.001

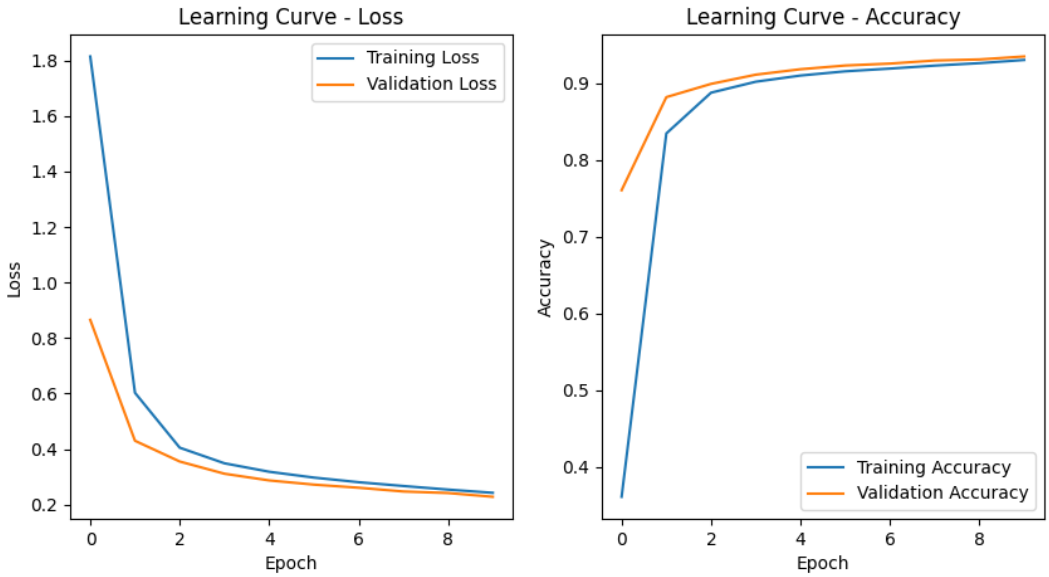


Figure 11: Alpha = 0.0001

3.5 Regularization Discussion

"Regularization is any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error." [4] L2 regularization, the type used in this project, is often called weight decay because it reduces the values of all weights. The effect of applying this repeatedly during each epoch decays or pulls weights towards zero. During each epoch gradient descent weights that minimize the loss function relative to their gradient. The combined effect of L2 regularization and gradient descent shrinks weights that aren't consistently contributing to reducing the loss. It also smooths out the loss surface, making it harder to get stuck in local minima and increasing the speed of the gradient descent. This implicit bias discourages large weights and helps keep models with a higher capacity from overfitting. Smaller values of the regularization constant α (referred to as λ in the textbook) have a stronger effect.

This experiment was run with similar conditions as the last one, except this time the value of α was varied from 1 to 1e-4 while the other hyperparameters were held constant. The 2-layer model was trained for 10 epochs on MINST, an 80/20 train/validation split, a batch size of 64, 1e-1 learning rate, 0.9 for momentum, and a hidden layer size of 128. The training, validating, and test cross-entropy accuracy was observed.

There was a clear trend towards higher training, validation, and test accuracy as the value of α decreased. With the best accuracy exhibited by the model with the smallest value of α. The learning curve for the model trained with $\alpha = 1$, which is effectively no regularization, is very stochastic with the large jumps in the loss and accuracy. As the α decreased the learning curves became increasingly smooth. The validation loss also got closer to the training loss, suggesting better generalization, which agrees with the testing accuracy.

3.6 Hyperparameter Tuning for Best Accuracy Data

Create a table summarizing the configuration of your best model and its performance:

Batch Size	**Learn. Rate**	**Hidden Size**	**Wall Clock (s)**	**Training Acc. (%)**	**Validation Acc. (%)**	**Test Acc. (%)**
64	0.01	384	351.71	0.9476	0.9488	0.9444
64	0.05	1536	721.16	0.9841	0.9720	0.9739
64	0.05	768	504.08	0.9856	0.9731	0.9743
64	0.05	384	352.12	0.9876	0.9738	0.9747
64	0.1	384	381.82	0.9934	0.9782	0.9782
64	0.5	384	344.05	0.9951	0.9792	0.9807
64	1	384	**335.84**	0.9931	**0.9798**	**0.9812**
32	0.01	384	425.41	0.9686	0.9629	0.9619
32	0.05	1536	918.75	0.9919	0.9746	0.9775
32	0.05	768	630.25	0.9930	0.9775	0.9783
32	0.05	384	422.23	0.9935	0.9773	0.9781
32	0.1	384	422.55	0.9953	0.9787	0.9800
32	0.5	384	409.80	0.9944	0.9802	0.9787
32	1	384	407.20	0.9899	0.9778	0.9787
16	0.01	384	564.26	0.9844	0.9721	0.9725
16	0.05	1536	1410.14	0.9946	0.9768	0.9800
16	0.05	768	939.44	0.9948	0.9794	0.9800
16	0.05	384	564.49	0.9953	**0.9798**	0.9801
16	0.1	384	561.56	**0.9958**	**0.9798**	0.9808
16	0.5	384	563.36	0.9911	0.9772	0.9805
16	1	384	546.33	0.9810	0.9701	0.9750

Table 3: Best Model Configuration and Accuracy. Epochs =200, Regularization = 0.0001, Momentum = 0.9

Hyperparameters for the model with highest test accuracy found in tuning:

- Batch Size = 64
- Learning Rate = 1
- Hidden Size = 384
- Regularization = 0.0001
- Momentum = 0.9
- Epoch = 200

3.7 Hyperparameter Tuning for Best Accuracy Discussion

In this experiment, I observed the wall clock time and the training, validation, and testing accuracies as I varied the values of batch size, learning rate, and hidden size. Only one hyperparameter was changed at a time. To help reduce the complexity of this experiment, I only varied the learning rate with respect to the models with a hidden size of 384. The other hyperparameters were held constant. The 2-layer model was trained for 200 epochs on MINST, an 80/20 train/validation split, 1e-4 L2 regularization, and 0.9 for momentum.

The time to converge, the wall clock time, decreased as the batch size got bigger, or as the hidden size got smaller, or as the learning rate got larger. A smaller batch size means that more complete passes through the network have to happen to cover all of the training data, requiring more compute. A smaller hidden size requires fewer overall operations because the matrices are smaller. The wall clock times for the same batch size and the same hidden size were all very similar. It's possible the larger learning rate resulted in larger weights earlier, which reduced the number of small floating point calculations required.

The learning rate seemed to have the largest impact on the overall accuracy of all the hyperparameters tested here. Larger learning rates resulted in the highest accuracies independent of the other hyperparameters. This helped the models avoid local minima.

The hidden size seemed to have the smallest impact on overall accuracy across the ranges tested here (384 to 1536). It seems that with this model architecture that 384 is sufficient capacity to represent the data and that additional parameters don't improve accuracy and add to the time to converge. The Minimum Description Length (MDL) principle says to favor the smallest possible model that is able to achieve a similar level of accuracy [5].

The impact of batch size is less straightforward. There seemed to be a general trend towards higher accuracies as the batch size decreased from 64 to 16. However, the best accuracy was very slightly higher (98.12% vs. 98.08%) with a model with a batch size of 64. Smaller batches can have more stochastic gradients because fewer samples are used to calculate them. This can result in noisier updates to the weights and help escape sharp local minima or saddle points. Batch size can act as a type of implicit regularization, with the nosier updates expressing a preference for exploration and flatter minima [6]. The noisiness of smaller batches can cause high learning rates to jump around more. This may explain why the model with the best accuracy had a larger batch size.

References

[1] Terence Parr and Jeremy Howard. "The Matrix Calculus You Need For Deep Learning". In: (2018). arXiv: 1802.01528 [astro-ph.IM]. URL: http://arxiv.org/pdf/1802.01528.

[2] Cheng Soon Ong Marc Peter Deisenroth A. Aldo Faisal. *Mathematics for Machine Learning*. Cambridge University Press, Apr. 2020. ISBN: 9781108569323, 1108569323.

[3] Matcha Latte. *Matrix in Latex*. Oct. 2014. URL: https://tex.stackexchange.com/questions/204621/matrix-in-latex.

[4] Aaron Courville Ian Goodfellow Yoshua Bengio. *Deep Learning*. MIT Press, Nov. 2016. ISBN: 9780262337373, 0262337371.

[5] Wikipedia Contributors. *Minimum Description Length*. Aug. 2023. URL: https://en.wikipedia.org/wiki/Minimum_description_length.

[6] Simon J.D. Prince. *Understanding Deep Learning*. MIT Press, Dec. 2023. ISBN: 9780262377102, 0262377101.